

Code Agents

Om Agrawal and Anirudh Khatry

Introduction

- Why study code generation with language models?
 - Structured format
 - Code can be compiled
 - Test cases can validate correctness
 - Large amounts of training data available through github
- We have also seen that code can be used for various downstream tasks, eg. in previous lectures we saw code can be use towards grounding robot actions, web-agent use, etc.

LiveCodeBench

Competition coding benchmark that evolves over time

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, Ion Stoica

Existing code generation benchmarks

- HumanEval (Mark Chen et. al., July 2021)
 - 164 benchmarks

```
def incr_list(l: list):  
    """Return list with elements incremented by 1.  
>>> incr_list([1, 2, 3])  
[2, 3, 4]  
>>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])  
[6, 4, 6, 3, 4, 4, 10, 1, 124]  
"""
```

Existing code generation benchmarks

- HumanEval (Mark Chen et. al., July 2021)
 - 100 benchmarks

```
def incr_list(l: list):  
    """Return list with elements incremented by 1.  
    >>> incr_list([1, 2, 3])  
    [2, 3, 4]  
    >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])  
    [6, 4, 6, 3, 4, 4, 10, 1, 124]  
    """  
    return [i + 1 for i in l]
```

Existing code generation benchmarks

- HumanEval (Mark Chen et. al., July 2021)
 - 100 benchmarks

```
def incr_list(l: list):  
    """Return list with elements incremented by 1.  
    >>> incr_list([1, 2, 3])  
    [2, 3, 4]  
    >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])  
    [6, 4, 6, 3, 4, 4, 10, 1, 124]  
    """  
    return [i + 1 for i in l]
```

- MBPP (Jacob Austin et. al., Aug 2021)
 - 500 benchmarks

prompt

Write a python function to check if a given number is one less than twice its reverse. Your code should satisfy these tests:

```
assert check(70) == False  
assert check(23) == False  
assert check(73) == True
```

Existing code generation benchmarks

- HumanEval (Mark Chen et. al., July 2021)
 - 100 benchmarks

```
def incr_list(l: list):  
    """Return list with elements incremented by 1.  
    >>> incr_list([1, 2, 3])  
    [2, 3, 4]  
    >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])  
    [6, 4, 6, 3, 4, 4, 10, 1, 124]  
    """  
    return [i + 1 for i in l]
```

- MBPP (Jacob Austin et. al., Aug 2021)
 - 500 benchmarks

prompt

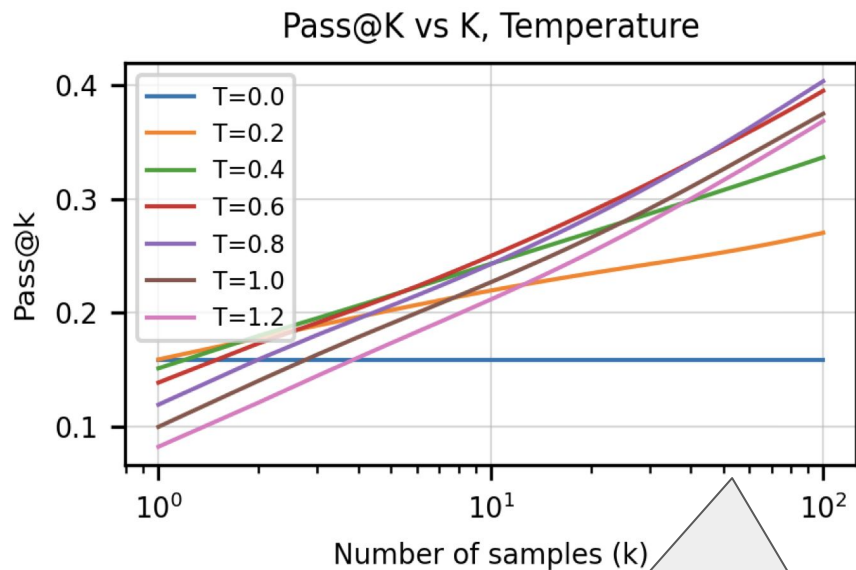
Write a python function to check if a given number is one less than twice its reverse. Your code should satisfy these tests:

```
assert check(70) == False  
assert check(23) == False  
assert check(73) == True
```

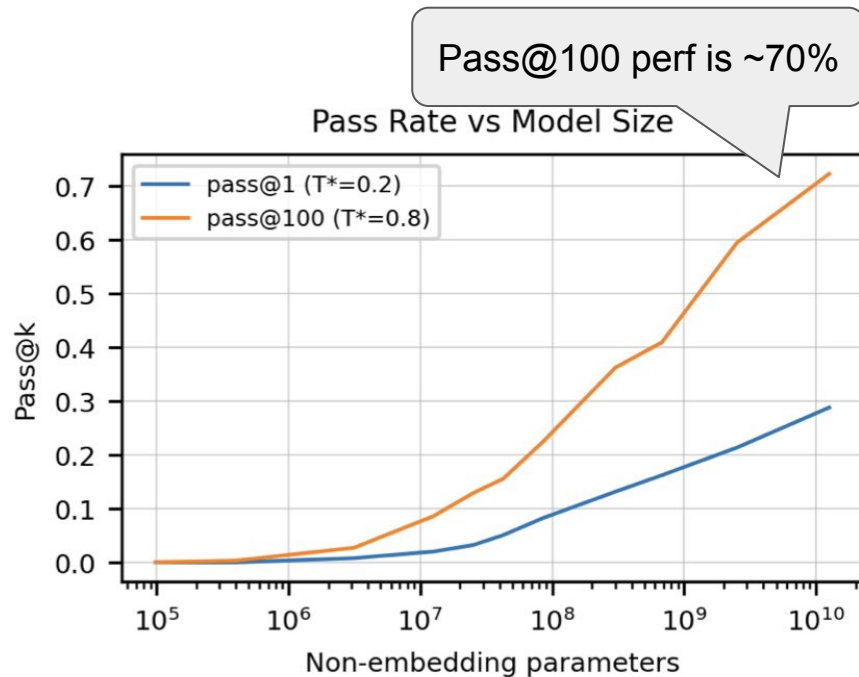
model

```
def check(n):  
    if n == 2*int(str(n)[::-1])-1:  
        return True  
    else:  
        return False
```

Model Performance (Then)



Sampling multiple candidates at higher temperature



Model Performance (Now)

Do we think LLMs are adept at NL to Code?

#	Model	pass@1
1	 01 Preview (Sept 2024) 	96.3
2	 01 Mini (Sept 2024) 	96.3
3	 GPT 4o (Aug 2024) 	92.7
4	Qwen2.5-Coder-32B-Instruct 	92.1
5	DeepSeek-V3 (Nov 2024) 	91.5
6	DeepSeek-V2.5 (Nov 2024) 	90.2
7	GPT-4-Turbo (April 2024) 	90.2
8	Gemini 1.5 Pro 002 	89
9	Grok Beta 	88.4
10	GPT 4o Mini (July 2024) 	88.4

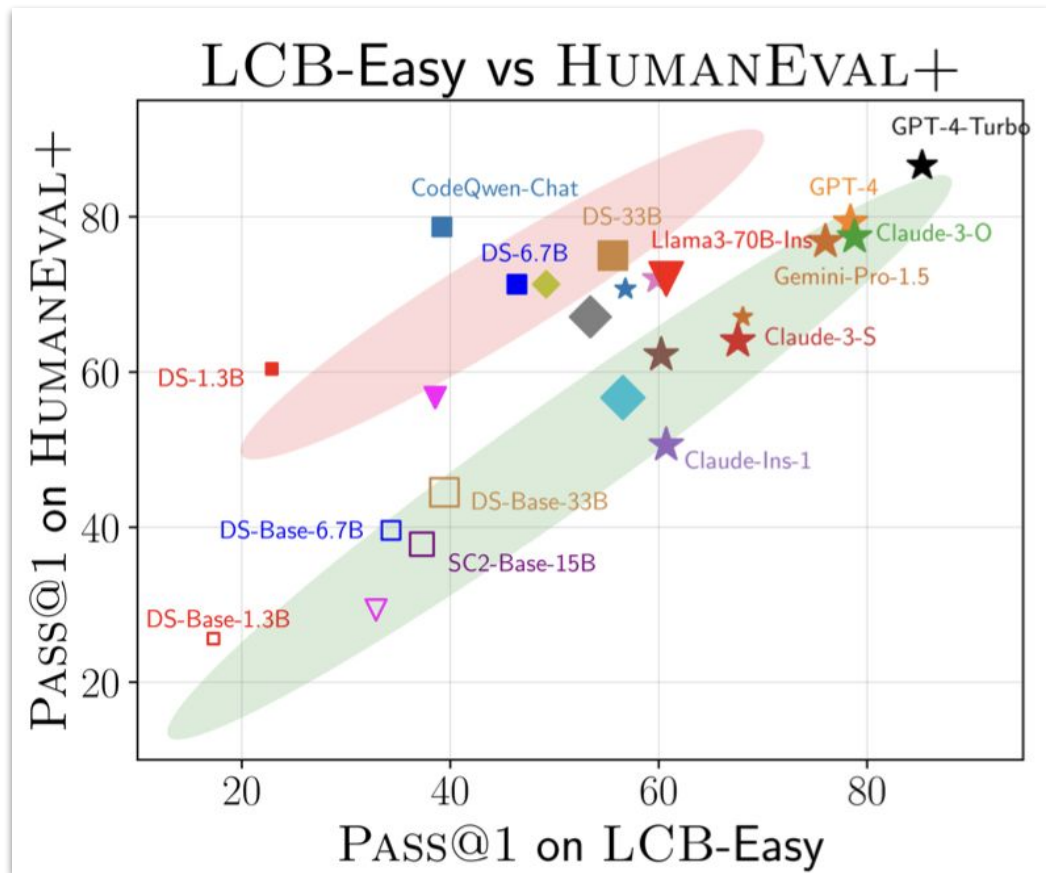
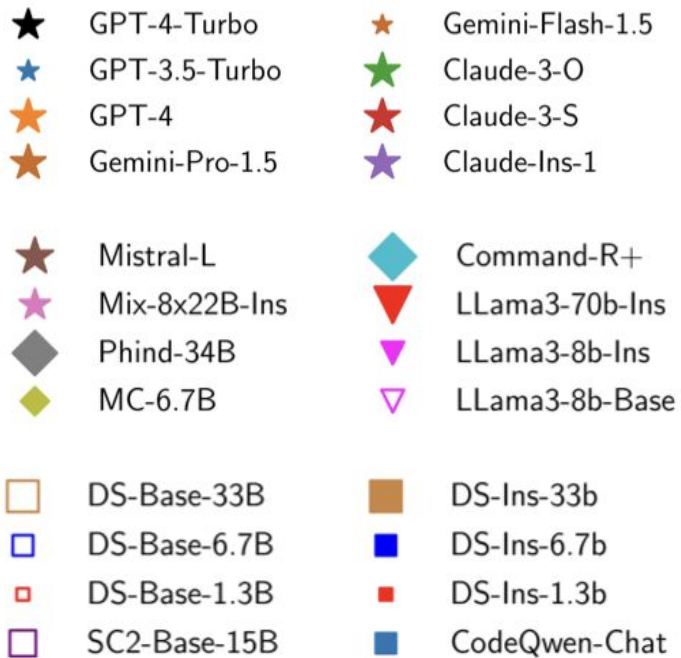
HumanEval

#	Model	pass@1
1	 01 Preview (Sept 2024) 	95.5
2	 01 Mini (Sept 2024) 	93.1
3	 Qwen2.5-Coder-32B-Instruct 	90.5
4	Gemini 1.5 Pro 002 	89.7
5	DeepSeek-Coder-V2-Instruct 	89.4
6	Claude Sonnet 3.5 (June 2024) 	89.4
7	claude-3-opus (Mar 2024) 	89.4
8	DeepSeek-V3 (Nov 2024) 	87.6
9	DeepSeek-V2.5 (Nov 2024) 	87.6
10	GPT 4o (Aug 2024) 	87.6

MBPP

<https://evalplus.github.io/leaderboard.html>

Contamination in data



Limitations of competition code benchmarks

- Data Contamination
- Limited Number of test cases
- Single Generation task
- Not stratified by difficulty

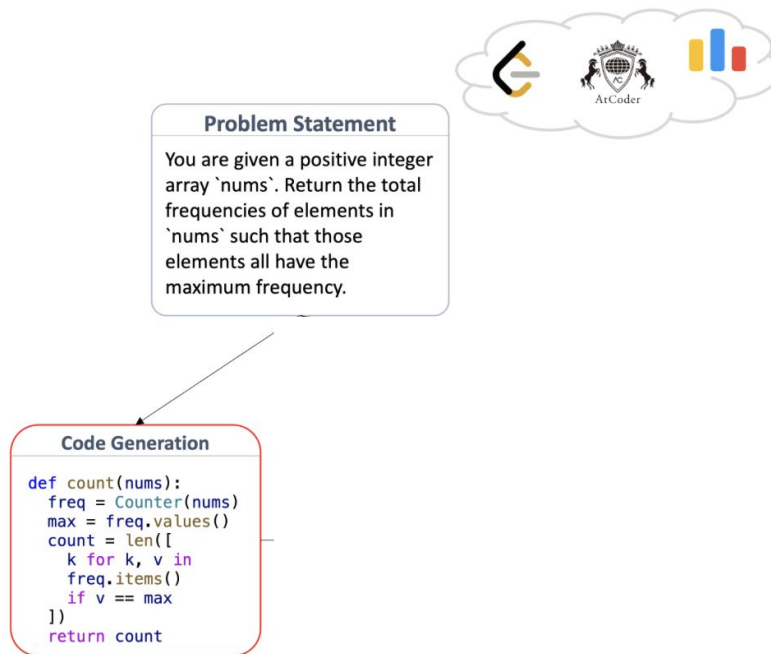
LiveCodeBench

RANK	MODEL	PASS@1	EASY	MEDIUM	HARD
1	O4-Mini (High)	80.2	99.1	89.4	63.5
2	O3 (High)	75.8	99.1	84.4	57.1
3	O4-Mini (Medium)	74.2	98.2	86.5	52.7
4	Gemini-2.5-Pro-06-05	73.6	99.1	87.2	50.2
5	DeepSeek-R1-0528	73.1	98.7	85.2	50.7
6	Gemini-2.5-Pro-05-06	71.8	98.2	82.3	50.2
7	EXAONE-4.0-32B	70	98.4	82.3	46.2
8	OpenReasoning-Nemotron-32B	69.8	98.3	81.4	46.3
9	O3-Mini-2025-01-31 (High)	67.4	99.1	84.4	38.4
10	OpenCodeReasoning-Nemotron-1.1-32B	66.8	97.9	79.6	41.1

<https://livecodebench.github.io/leaderboard.html>

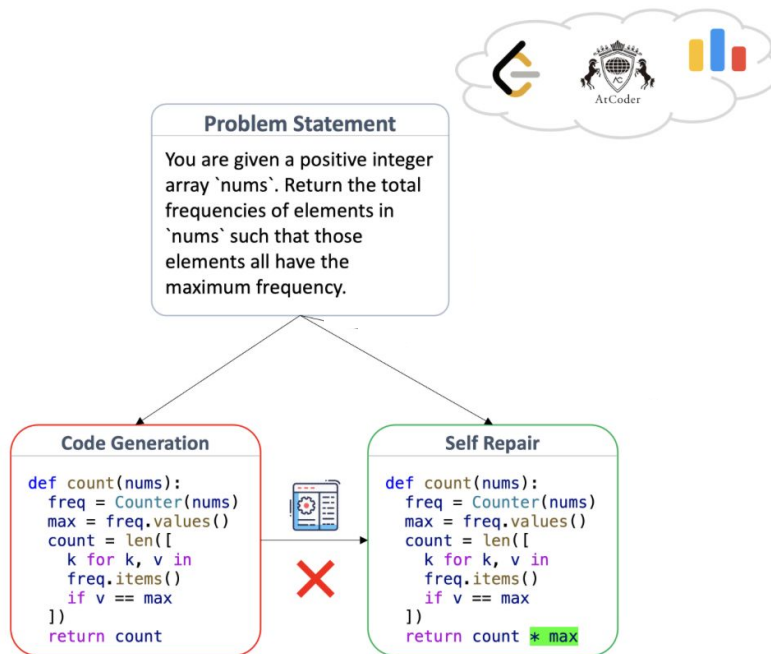
LiveCodeBench supports multiple tasks

1. Code Generation



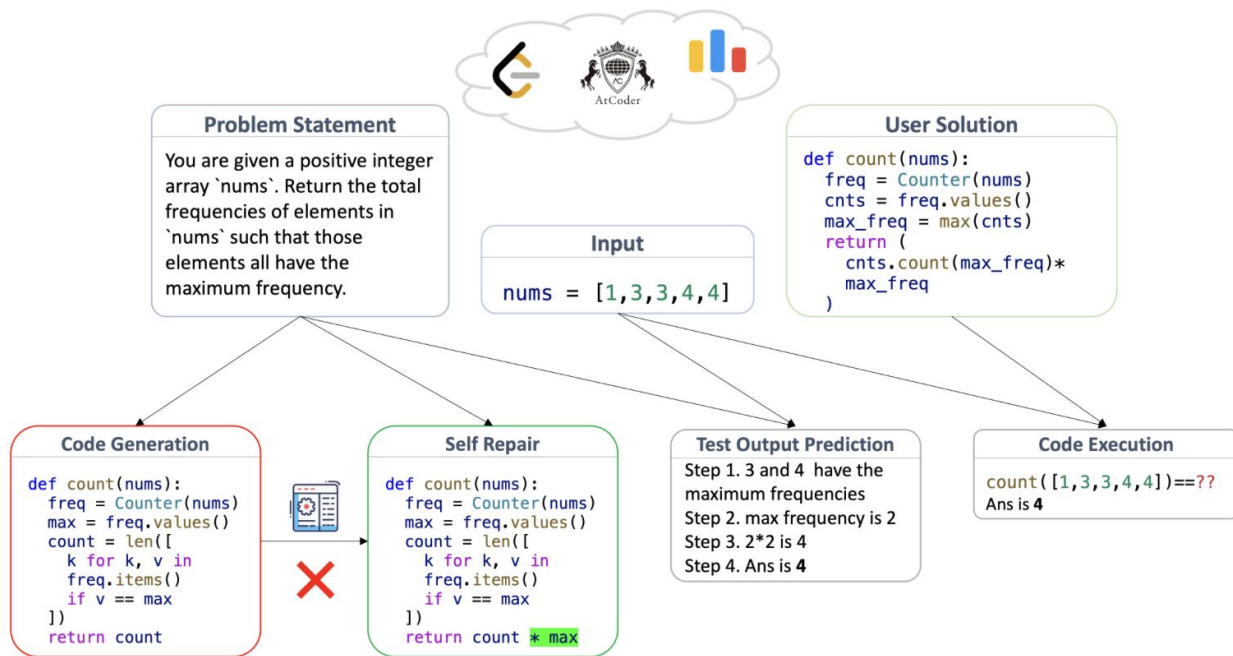
LiveCodeBench supports multiple tasks

1. Code Generation
2. Code Repair

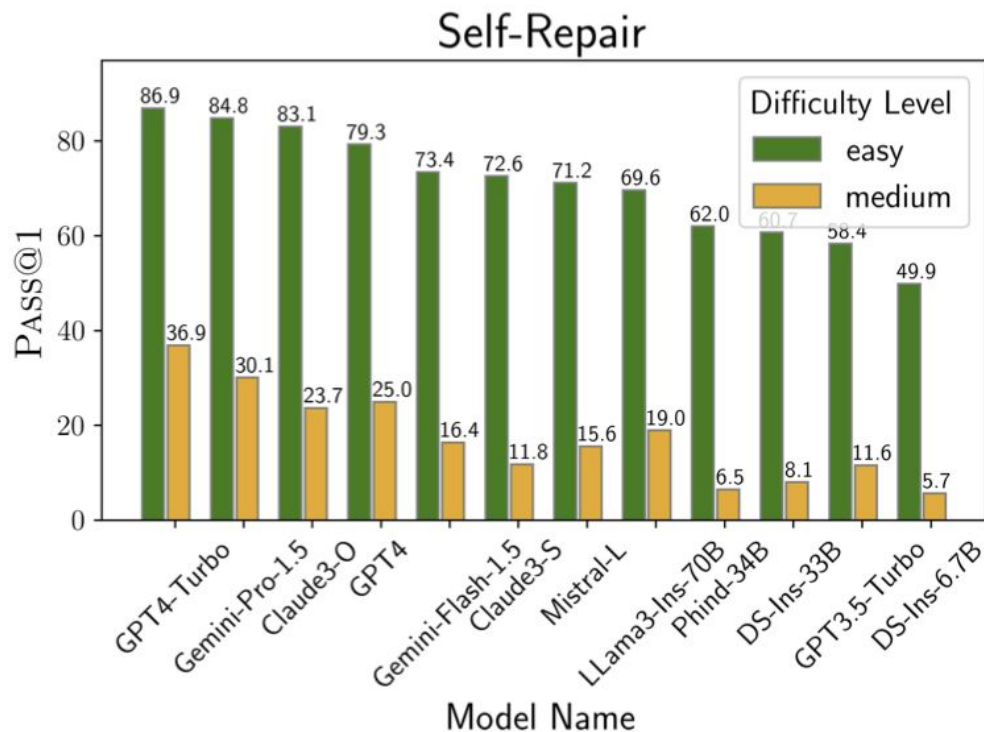


LiveCodeBench support multiple tasks

1. Code Generation
2. Code Repair
3. Code Execution



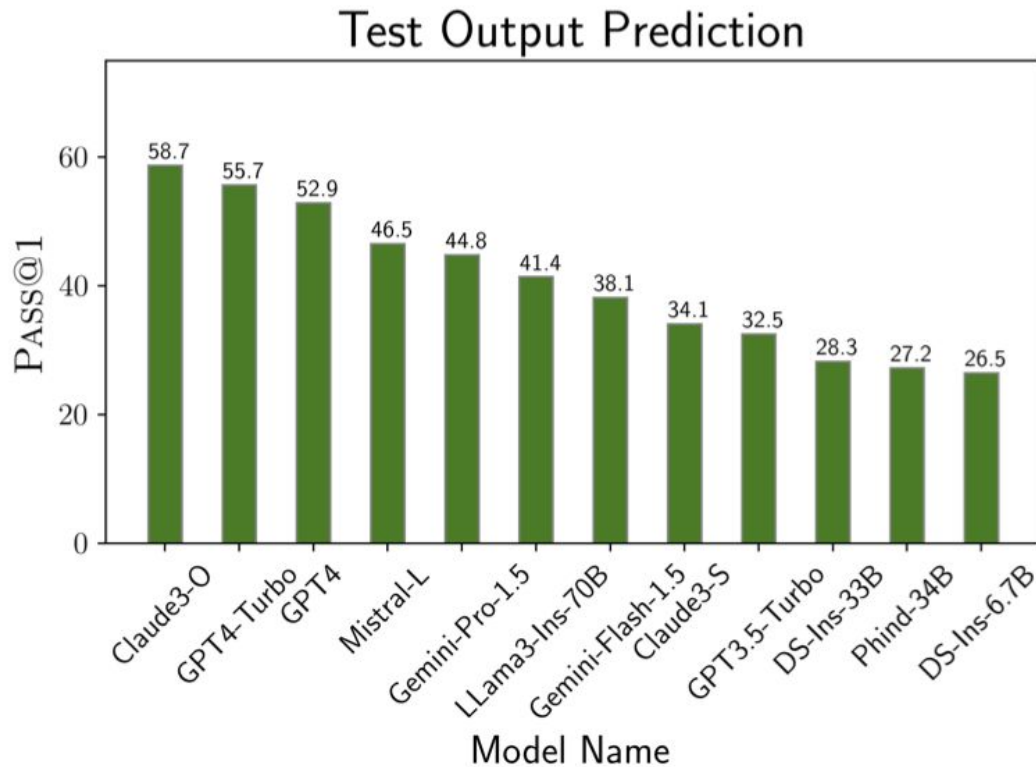
Results: Repair



Results: Code Execution



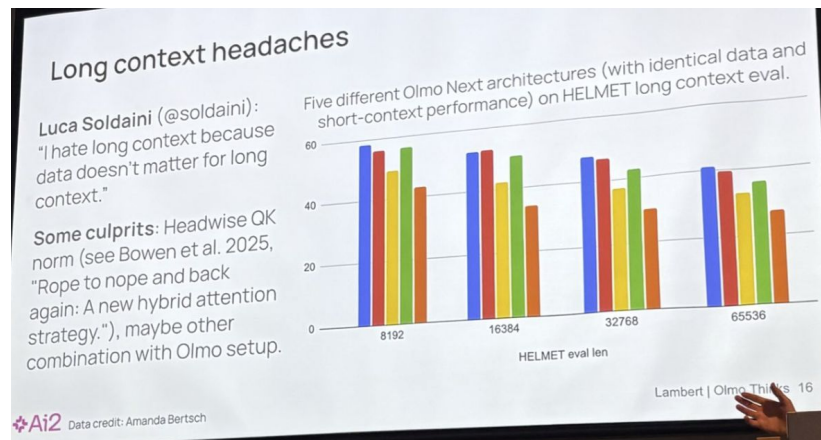
Results: Test Output Prediction



But these are small function level generation tasks

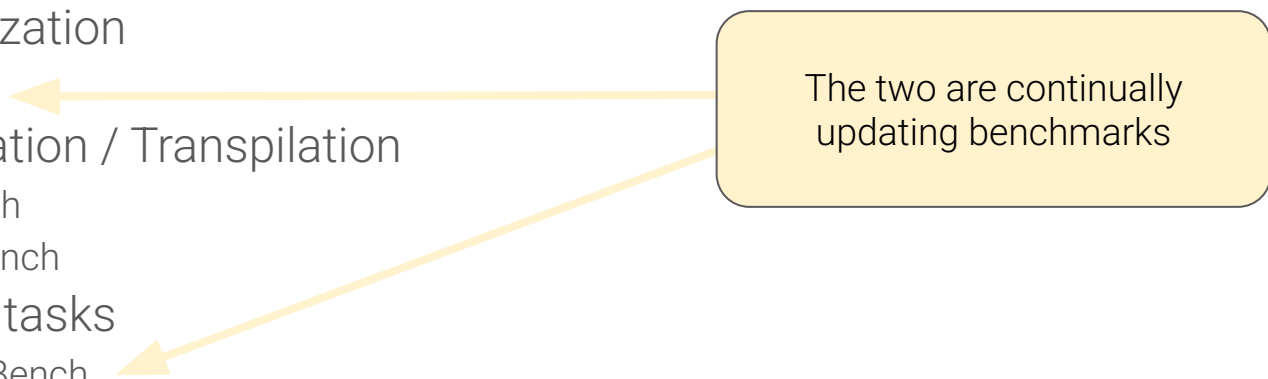
- Most applications deal with repository level code
 - Multiple files
 - Cross file dependencies
 - Version controlled format

- We cannot fit the entire project in the LLM's context window



Repository level software engineering tasks

- Code editing
 - SWE bench
 - SWE bench M
- Code Optimization
 - GSO
- Code Generation / Transpilation
 - RepoBench
 - CRUST-Bench
- Open-ended tasks
 - Terminal Bench



The two are continually updating benchmarks

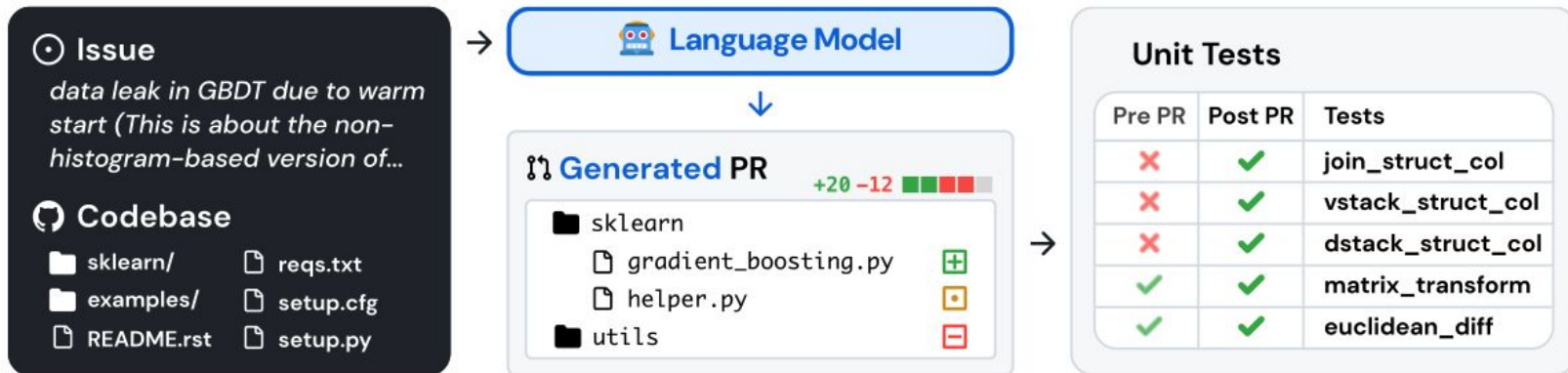
SWE-Bench

can language models resolve real-world github issues?

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, Karthik Narasimhan

Motivation

- In the real world, software engineering is not simple:
 - Navigate large codebase
 - Interplay between functions in different files
 - Spotting error in convoluted code



How do you build a Software Engineering benchmark

1

🔗 Scrape PRs

🔄 12 popular repositories

🐍 >90% Python Code

collecting PRs from **12**
popular open-source
Python repositories on
GitHub, producing about
~90,000 PRs in total

How do you build a Software Engineering benchmark

1 🗑️ Scrape PRs

- 🔄 12 popular repositories
- 🐍 >90% Python Code

collecting PRs from **12**
popular open-source
Python repositories on
GitHub, producing about
~90,000 PRs in total

2 ⚙️ Attribute Filter

- ✓ Resolves an issue
- ✓ Contributes tests

Accepted PR, Added tests

How do you build a Software Engineering benchmark

1 🗄️ Scrape PRs

- 🔄 12 popular repositories
- 🐍 >90% Python Code

collecting PRs from **12** popular open-source Python repositories on GitHub, producing about **~90,000** PRs in total

2 ⚙️ Attribute Filter

- ✓ Resolves an issue
- ✓ Contributes tests

Accepted PR, Added tests

3 🧪 Execution Filter

- ✓ Installs successfully
- ✓ PR passes all tests

Passes all tests on repository in the same environment

Prompt Format

You will be provided with a partial code base and an issue statement explaining a problem to resolve.

```
<issue>
{ISSUE TEXT}
</issue>
```

```
<code>
...
</code>
```

Here is an example of a patch file. It consists of changes to the code base. It specifies the file names, the line numbers of each change, and the removed and added lines. A single patch file can contain changes to multiple files.

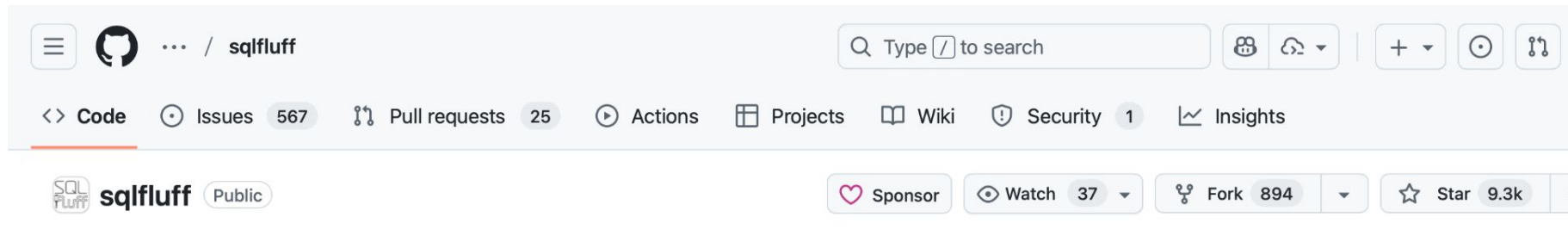
```
<patch>
...
</patch>
```

I need you to solve the provided issue by generating a single patch file that I can apply directly to this repository using git apply. Please respond with a single patch file in the format shown above. Respond below:

SWE-bench

- SWE bench offers several advantages over existing LM benchmarks
 - Realistic setting that uses user-submitted issues and solutions

2,294 SE Tasks over 12 repositories



SWE-bench

- SWE bench offers several advantages over existing LM benchmarks
 - Realistic setting that uses user-submitted issues and solutions
 - Diverse Inputs

SQLFluff

```
[CLI](https://docs.sqlfluff.com/en/stable/cli.html), not to limit it further. It would be great to have an option to further limit the amount of prints when running `sqlfluff fix`, especially in combination with deployment using a pre-commit hook. For example, only print the return status and the number of fixes applied, similar to how it is when using `black` in a pre-commit hook. This hides the potentially long list of fixes that are being applied to the SQL files, which can get quite verbose.
```

What Happened ...

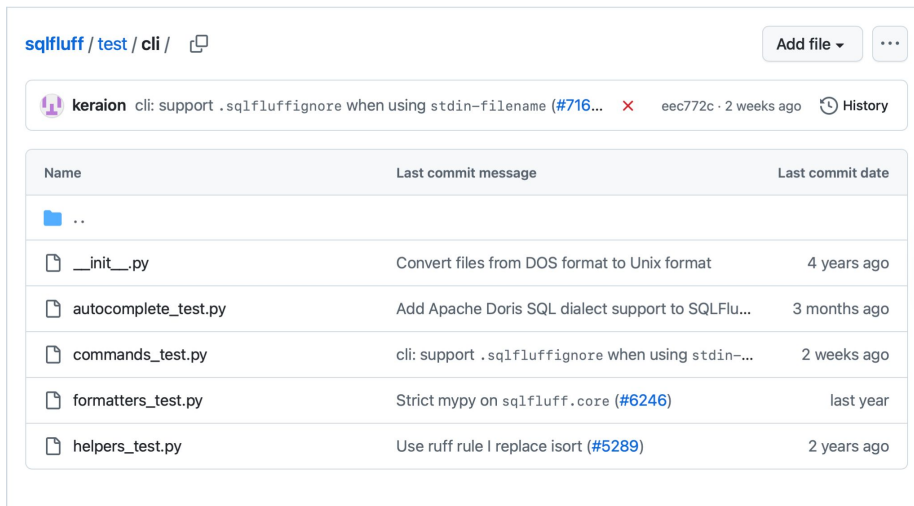
...

```
### Expected Behaviour at the end of the file, that is after `{%- endif %}` instead of adding the new line at the end of the SQL query - so the result should look like this: `` `{% if true %} SELECT 1 + 1 {%- endif %}` ``
```

Observed Behaviour Adds a new line to the end of the SQL query instead of in the end of the file.

SWE-bench

- SWE bench offers several advantages over existing LM benchmarks
 - Realistic setting that uses user-submitted issues and solutions
 - Diverse inputs
 - Robust framework for execution based evaluation



The screenshot shows the GitHub repository page for `sqlfluff`, specifically the `test/cli` directory. At the top, the repository path is `sqlfluff / test / cli`. Below this, a commit by user `keraion` is highlighted, with the message `cli: support .sqlfluffignore when using stdin-filename` and a link to pull request `#716`. The commit was made 2 weeks ago. Below the commit information is a table listing files in the directory.

Name	Last commit message	Last commit date
..		
<code>__init__.py</code>	Convert files from DOS format to Unix format	4 years ago
<code>autocomplete_test.py</code>	Add Apache Doris SQL dialect support to SQLFlu...	3 months ago
<code>commands_test.py</code>	cli: support .sqlfluffignore when using stdin-...	2 weeks ago
<code>formatters_test.py</code>	Strict mypy on <code>sqlfluff.core</code> (#6246)	last year
<code>helpers_test.py</code>	Use ruff rule I replace isort (#5289)	2 years ago

Results

Model	SWE-bench		% Resolved	% Apply
	% Resolved	% Apply		
Claude 3 Opus	3.79	46.56	4.33	51.67
Claude 2	1.97	43.07	3.00	33.00
ChatGPT-3.5	0.17	26.33	0.33	10.00
GPT-4-turbo	1.31	26.90	2.67	29.67
SWE-Llama 7b	0.70	51.74	1.33	38.00
SWE-Llama 13b	0.70	53.62	1.00	38.00

<5%
performance in
2023

Model	% Resolved	Org	Date	Site
✓ SWE-agent 1.0 (Claude 3.7 Sonnet)	33.83		2025-02-27	Link
✓ OpenHands + CodeAct v2.1 (claude-3.5-sonnet-20241022)	29.38		2024-11-03	Link
AutoCodeRover-v2.0 (Claude-3.5-Sonnet-20241022)	24.89		2024-11-21	Link
✓ SWE-agent + Claude 3.5 Sonnet	18.13		2024-06-20	-
✓ SWE-agent + GPT 4 (1106)	12.47		2024-04-02	Link
✓ SWE-agent + GPT 4o (2024-05-13)	11.99		2024-07-28	Link
✓ SWE-agent + Claude 3 Opus	10.51		2024-04-02	-
✓ RAG + Claude 3 Opus	3.79		2024-04-02	Link

33%
performance
with Agents

Limitations

- Only uses python
- Task instance codebases tend to be structured similarly because every repository is a PyPI package
- Problem statement does not include images
- Edits are fairly concise

SWE-bench Multimodal

Do AI Systems Generalize to Visual Software Domains?

John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida I. Wang, Ofir Press

SWE-bench Multimodal

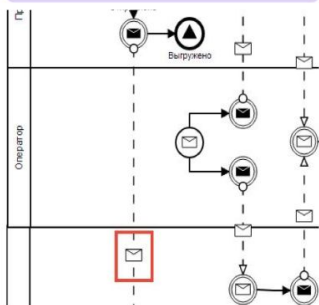
- “Do AI Systems Generalize to Visual Software Domains?”
 - E.g front end web development

Diagramming

Show message element name

Currently, names of message elements on message flows are not rendered

Given this example diagram
[Image] ...



 bpmn-js

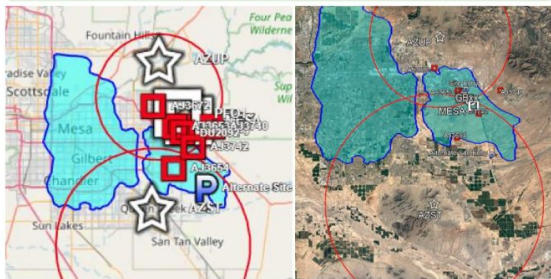
Interactive Mapping

KML Symbol Align/Placement/Size

There is a bug with the anchor point for some symbols

I've attached a screen clipping from Google Earth to show how it is supposed to look.

[Right Image] ...



 openlayers

Syntax Highlighting

Bracket highlighted with different color in class inheritance context.

- Reproduced in JSFiddle: <https://jsfiddle.net/kkangmj/e7h48w36/7/>

[Image] ...

```
open class Tag

class TABLE: Tag {
  fun tr(init: TR.() -> Unit)
}

class TR: Tag {
  fun td(init: TD.() -> Unit)
}

class TD: Tag
```

 highlight.js

Web Frameworks

[CascaderSelect]使用虚拟滚动时背景异常

Component: CascaderSelect

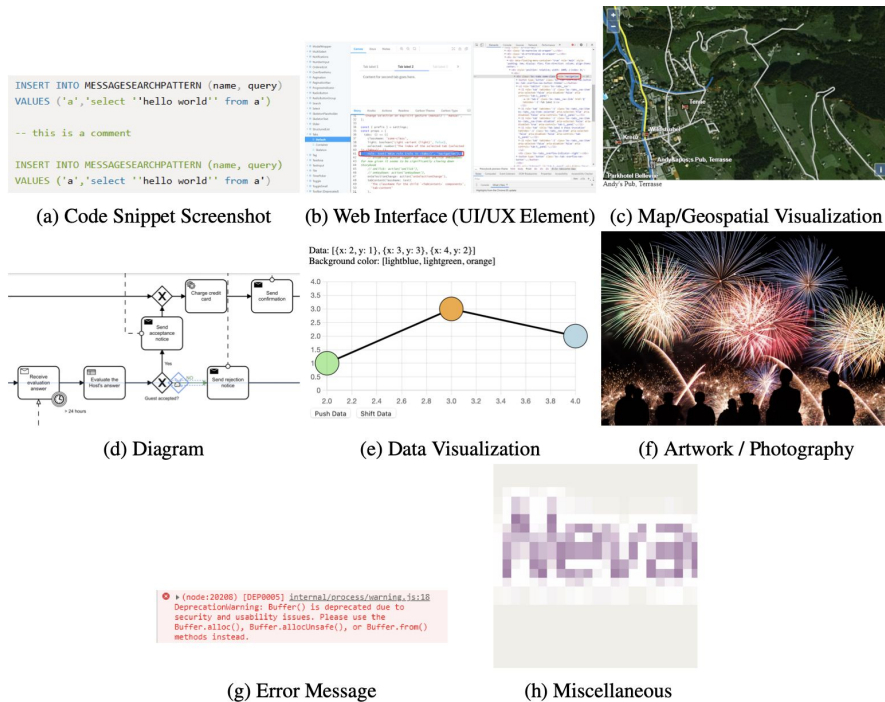
Steps to reproduce

[Image] ...



 next

Multimodal inputs reduce downstream performance



Results

System	Model	% Resolved	\$ Avg. Cost
SWE-agent M	GPT-4o	12.2	2.94
	Claude 3.5 Sonnet	11.4	3.11
SWE-agent JS	GPT-4o	9.2	0.99
	Claude 3.5 Sonnet	12.0	3.11
SWE-agent Base	GPT-4o	12.0	2.07
	Claude 3.5 Sonnet	12.2	1.52
Agentless JS	GPT-4o	3.1	0.38
	Claude 3.5 Sonnet	6.2	0.42
RAG	GPT-4o	6.0	0.17
	Claude 3.5 Sonnet	5.0	0.15

Model	% Resolved
  GUIRepair + o3 (2025-04-16)	35.98
Refact.ai Agent	35.59
 OpenHands-Versa (Claude-Sonnet 4)	34.43
 GUIRepair + o4-mini (2025-04-16)	33.85
 OpenHands-Versa (Claude-3.7 Sonnet)	31.33
 GUIRepair + GPT 4.1 (2025-04-14)	31.14
 GUIRepair + GPT 4o (2024-08-06)	30.37
 Agentless Lite + Claude-3.5 Sonnet	25.34
 SWE-agent Multimodal + GPT 4o (2024-08-06)	12.19

Analysis / Takeaways

- Resolving SWE-bench M issues requires improved visual understanding
- Localization modules are overly engineered for Python
 - They had to adapt many existing systems to work with SWE-bench M
- Multimodal tools increase task complexity but can improve agent performance

RepoBench

Benchmarking Repository-Level Code Auto-Completion Systems

Tianyang Liu, Canwen Xu, Julian McAuley

Background

- Previous work focuses on code completion within the scope of a single file
- Eval datasets and benchmarks fail to recognize the importance of multi-file projects

RepoBench

- benchmark for evaluating the effectiveness of repository-level code auto-completion systems
- 3 distinct eval subtasks
 - (1) The Retrieval Task (RepoBench-R)
 - (2) The Code Completion Task (RepoBench-C)
 - (3) The End-to-End Pipeline Task (RepoBench-P)
 - Combination of first two

Results - RepoBench-R

5-9 candidates

≥ 10 candidates

Jaccard Similarity
Outperforms Edit
Similarity

Superior Performance
of UniXcoder

Python Retrieval
Shows Higher
Accuracy Than Java

Table 2: Baseline results of RepoBench-R on Python and Java retrieval tasks for *Easy* and *Hard* subset. The models we use are codebert-base for CodeBERT, unixcoder-base for UniXcoder.

Retrieval	Model	Easy				Hard					
		XF-F		XF-R		XF-F			XF-R		
		acc@1	acc@3	acc@1	acc@3	acc@1	acc@3	acc@5	acc@1	acc@3	acc@5
Python	Random	15.68	47.01	15.61	46.87	6.44	19.28	32.09	6.42	19.36	32.19
	Lexical	20.82	53.27	24.28	54.72	10.01	25.88	39.88	11.38	26.02	40.28
	Lexical	17.91	50.61	20.25	51.73	7.68	21.62	36.14	8.13	22.08	37.18
	Semantic	16.47	48.23	17.87	48.35	6.56	19.97	33.34	7.03	19.73	32.47
	Semantic	25.94	59.69	29.40	61.88	17.70	39.02	53.54	20.05	41.02	54.92
Java	Random	15.33	45.98	15.43	46.13	5.59	16.88	28.15	5.64	16.91	28.18
	Lexical	15.33	46.73	19.08	52.00	7.15	19.92	32.12	9.22	22.65	34.17
	Lexical	14.67	44.99	16.23	47.78	5.80	17.55	28.73	6.67	17.80	28.62
	Semantic	15.38	46.77	16.27	45.98	6.07	17.42	28.94	5.92	17.73	28.53
	Semantic	17.34	50.52	24.15	57.83	10.79	26.42	39.73	15.10	33.38	46.17

Results - RepoBench-C

Table 3: RepoBench-C performance of CodeGen models with varying parameter sizes (350M to 16B), including original and fine-tuned (FT) versions, StarCoder and Codex (code-davinci-002), across Python and Java, evaluated using Exact Match (EM), Edit Similarity (ES). 'All' represents the average performance over the mixture of all test data, weighted by the size of each test setting.

	Model	Params.	XF-F		XF-R		IF		All	
			EM	ES	EM	ES	EM	ES	EM	ES
Python	CodeGen	350M	15.14	60.18	27.72	68.91	25.24	67.80	20.71	64.22
	CodeGen+FT	350M	16.19	62.82	27.88	69.64	23.81	66.83	20.85	65.41
	CodeGen	2.7B	22.09	64.94	34.48	72.67	31.27	70.93	27.35	68.30
	CodeGen+FT	2.7B	22.38	67.20	33.72	73.69	28.94	69.90	26.66	69.34
	CodeGen	6.1B	26.98	67.95	38.30	74.58	34.96	72.59	31.67	70.68
	StarCoder	15.5B	28.06	69.64	37.28	73.69	33.86	72.37	31.67	71.28
	CodeGen	16.1B	28.86	68.68	38.48	73.71	37.60	73.78	33.41	71.22
	Codex	175B	26.67	70.29	39.40	76.39	33.47	72.54	31.31	72.22
Java	CodeGen	350M	12.89	59.64	24.20	66.31	33.33	68.60	21.21	63.64
	CodeGen+FT	350M	16.12	65.10	25.88	72.08	33.91	71.57	23.34	68.44
	CodeGen	2.7B	20.77	66.65	30.06	69.96	39.97	72.93	28.31	69.17
	CodeGen+FT	2.7B	20.64	68.64	31.24	74.76	38.99	74.65	28.20	71.67
	CodeGen	6.1B	22.34	68.37	31.88	71.13	40.40	72.96	29.59	70.28
	StarCoder	15.5B	30.17	75.10	39.46	78.25	48.14	79.39	37.35	77.01
	CodeGen	16.1B	23.42	68.53	33.08	70.99	40.61	72.85	30.45	70.30
	Codex	175B	35.16	78.05	45.58	82.09	53.09	82.20	43.14	80.22
Python	StarCoder	15.5B	23.00	66.29	33.01	72.85	29.00	68.79	26.84	68.39
	Codex	175B	28.02	69.89	40.51	76.66	33.19	71.94	32.13	71.90
Java	StarCoder	15.5B	20.10	66.14	30.39	70.87	41.23	73.29	28.41	69.21
	Codex	175B	33.07	75.63	42.76	79.97	51.68	80.60	40.52	77.98

Pronounced
Performance
Differences in Java

Comparable
Performance in
Python

Substantial
Performance Gap

Results - RepoBench-P

Table 4: Comparison of various retrieval strategies on the RepoBench-P for Python and Java using Codex [9] (code-davinci-002). Each strategy is evaluated in terms of Exact Match (EM) and Edit Similarity (ES) metrics for XF-F, XF-R, and IF settings. ‘All’ represents the average performance over the mixture of all test data, weighted by the size of each test setting. Strategies (*Gold-Only* and *Gold-Filled*), marked with an asterisk (*), include gold snippets for benchmarking purposes and serve only as references; they do not embody oracle capabilities.

Retrieval Method		XF-F		XF-R		IF		All	
		EM	ES	EM	ES	EM	ES	EM	ES
Python	Gold-Only*	30.59	70.43	40.65	74.45	41.10	78.32	35.79	73.59
	Gold-Filled-Head*	31.07	70.48	39.77	74.42	41.87	78.56	36.07	73.68
	Gold-Filled-Tail*	31.35	70.73	40.56	74.37	41.21	78.50	36.18	73.77
	UniXcoder-H2L	30.99	70.68	40.71	74.74	43.19	79.18	36.61	74.02
	UniXcoder-L2H	32.12	71.36	40.59	74.48	43.07	79.22	37.11	74.32
	Random	28.06	68.95	38.75	73.81	41.16	78.34	34.15	72.72
	Baseline	26.75	68.16	37.60	73.30	40.79	78.26	33.15	72.20
Java	Gold-Only*	32.48	70.39	43.51	76.49	55.91	81.65	41.62	74.95
	Gold-Filled-Head*	32.48	70.29	43.44	76.36	55.84	81.68	41.59	74.88
	Gold-Filled-Tail*	32.37	70.30	43.48	76.63	55.66	81.55	41.49	74.91
	UniXcoder-H2L	32.46	70.16	42.79	76.14	56.71	81.86	41.70	74.83
	UniXcoder-L2H	32.69	70.33	43.08	76.21	56.57	81.89	41.83	74.93
	Random	31.34	69.81	42.08	75.73	55.94	81.67	40.77	74.51
	Baseline	30.73	69.48	41.44	75.42	56.18	81.70	40.40	74.29

Effective Retrieval
Enhances
Performance

GSO

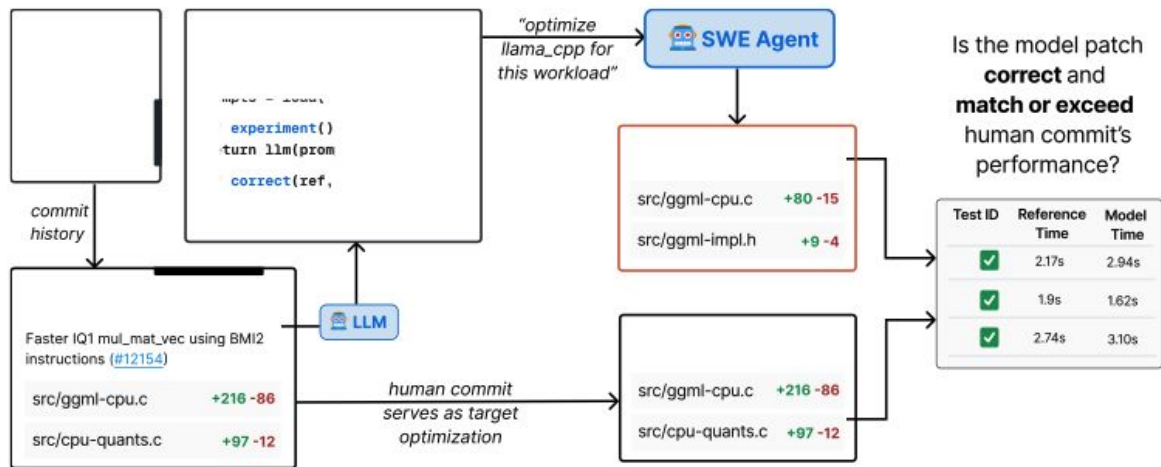
Challenging Software Optimization Tasks for Evaluating SWE-Agents

Manish Shetty, Naman Jain, Jinjian Liu, Vijay Kethanaboyina, Koushik Sen, Ion Stoica

GSO

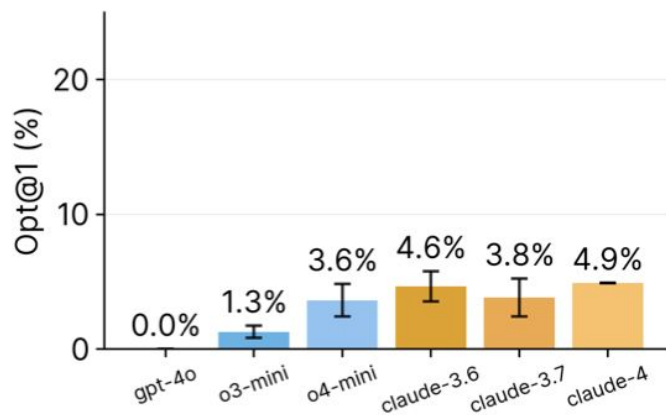
- “Can LLM agents aid in the development of high-performance software?”
- a new benchmark designed to test how well SWE-Agents can optimize real-world software for better performance
- Instead of just fixing bugs or writing small programs, GSO focuses on making code run faster
 - much harder and more practical challenge

Example of GSO task



They develop an automated pipeline that generates performance tests and analyzes repository commit history to identify real-world code optimization tasks

Results



RANK	MODEL	SETTING	SCORE	DATE	ORG
1	 Claude-3.5-V2-Sonnet + OpenHands	Opt@10	15.7%	May 30, 2025	GSO ↗
2	 O4-Mini (high) + OpenHands	Opt@10	12.7%	May 30, 2025	GSO ↗
3	 O3 (high) + OpenHands	Opt@1	8.8%	Jun 9, 2025	GSO ↗
4	 GPT-5 (high) + OpenHands	Opt@1	6.9%	Aug 10, 2025	GSO ↗
4	 Claude-4-Opus + OpenHands	Opt@1	6.9%	Aug 10, 2025	GSO ↗
6	 Qwen3-Coder + OpenHands	Opt@1	4.9%	Sep 2, 2025	GSO ↗
6	 Kimi-K2-Instruct + OpenHands	Opt@1	4.9%	Aug 26, 2025	GSO ↗
6	 Claude-4-Sonnet + OpenHands	Opt@1	4.9%	May 30, 2025	GSO ↗
9	 Claude-3.5-V2-Sonnet + OpenHands	Opt@1	4.6%	May 30, 2025	GSO ↗

Contributions

- Automated Benchmark Creation
- Precise Task Specification
- Rigorous Evaluation
 - introduces a new metric (OPT@K) that compares the agent's optimization to the human's, focusing on relative speedup and correctness, not just raw speed.
- Comprehensive Analysis
 - The authors tested leading AI models (like GPT-4O, Claude, etc.) and found that even the best agents succeed on less than 5% of tasks
 - Discussed future research directions

Findings

- Current AI Agents Struggle, especially for low-level or multi-language tasks
- Common failure modes
 - Avoid or mishandle low level code (e.g. C/C++)
 - Favor lazy optimizations - like tweaking compiler flags
 - Often misdiagnose root cause
- Some successes though
 - Agents sometimes do find clever optimizations, although it is rare

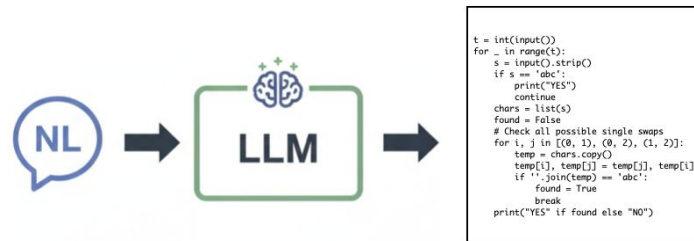
CRUST-Bench

A Comprehensive Benchmark for C-to-safe-Rust Transpilation

Anirudh Khatry, Robert Zhang, Jia Pan, Ziteng Wang, Qiaochu Chen, Greg Durrett, Isil Dillig

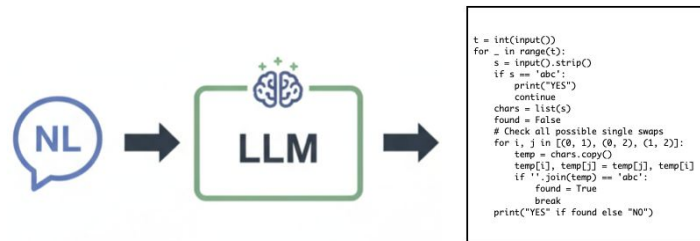
Problem

- Competition Coding
 - Input: Short specification
 - Output: Short solution



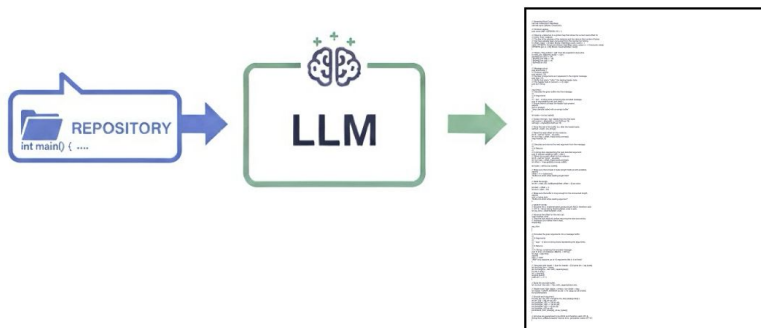
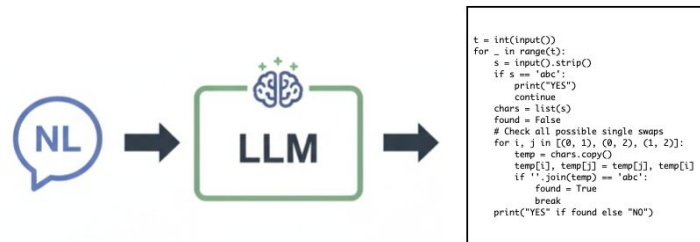
Problem

- Competition Coding
 - Input: Short specification
 - Output: Short solution
- Bug Fixing
 - Input: Entire repo
 - Output: Short patch



Problem

- Competition Coding
 - Input: Short specification
 - Output: Short solution
- Bug Fixing
 - Input: Entire repo
 - Output: Short patch
- Repository Level Transpilation
 - Input: Entire repository
 - Output: Entire repository with the same functionality



Result

C Source

```
value_t* find_value(  
    node_t* self,  
    void* key,  
    size_t key_len) {  
    node_t *curr = self;  
    while (curr) {  
        int cmp = key_cmp(...);  
        if (cmp == 0)  
            return curr->value;  
        else if (cmp < 0)  
            curr = curr->left;  
        else  
            curr = curr->right;  
    }  
    return NULL;  
}
```

Rust Interface

```
impl Node {  
    fn find_value(  
        &self,  
        key_hash: u32,  
        key: Vec<u8>,  
        key_len: usize)  
        -> Option<&Value> {  
        unimplemented!()  
    }  
    ...  
}
```

Rust Tests

```
#[test]  
fn test_find() {  
    let value: Option<&Value> =  
        node.find_value(  
            key_hash,  
            &int_key,  
            key_len  
        )  
    assert(value.exists())  
    ...  
}
```

Result

C Source

```
value_t* find_value(
    node_t* self,
    void* key,
    size_t key_len) {
    node_t *curr = self;
    while (curr) {
        int cmp = key_cmp(...);
        if (cmp == 0)
            return curr->value;
        else if (cmp < 0)
            curr = curr->left;
        else
            curr = curr->right;
    }
    return NULL;
}
```

Rust Interface

```
impl Node {
    fn find_value(
        &self,
        key_hash: u32,
        key: Vec<u8>,
        key_len: usize)
        -> Option<&Value> {
        unimplemented!()
    }
    ...
}
```

Rust Tests

```
#[test]
fn test_find() {
    let value: Option<&Value> =
        node.find_value(
            key_hash,
            &int_key,
            key_len
        )
    assert(value.exists())
    ...
}
```



Transpile

```
impl Node {
    pub fn find_value(&self,
        key_hash: u32,
        key: Vec<u8>,
        key_len: usize)
        -> Option<Value> {
        if let Some(ref right) = self.child_right
        {
            return right.lock().ok()?.unwrap()
        } ...
    } ...
}
```

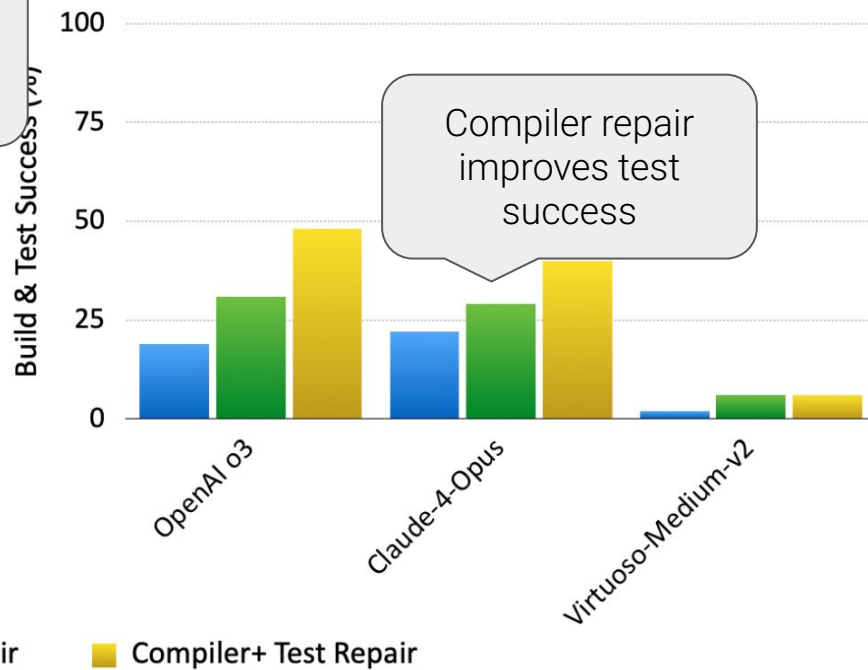
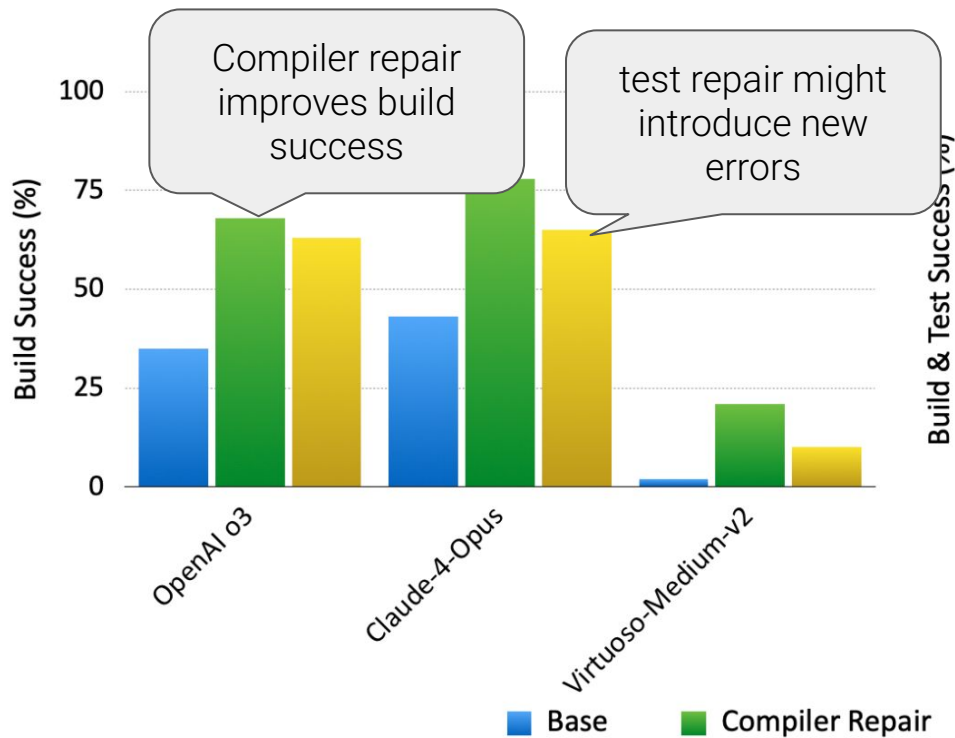
Rust Implementation

Run tests

Compile



Results



Agents for code

Agents for repository level settings

Requirements:

1. Need to account for multiple files
2. Global context
3. Testing mechanism

Motivations

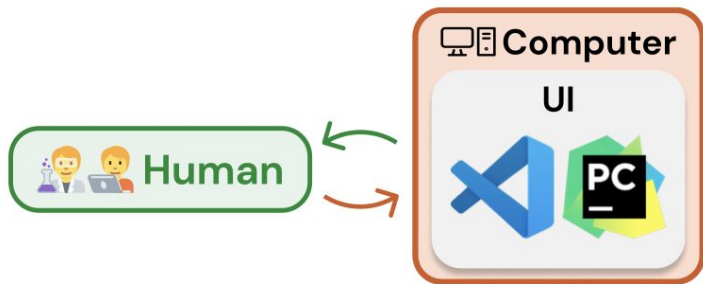
Designing a good agent must involve:

- Simple and easy to understand actions
 - Subset of terminal commands
- Short and compact actions
 - edit line:line
- Environment should give concise feedback
 - Irrelevant information must be filtered (information bloat)
- Guardrails improve error recovery
 - Syntax checkers, linting etc

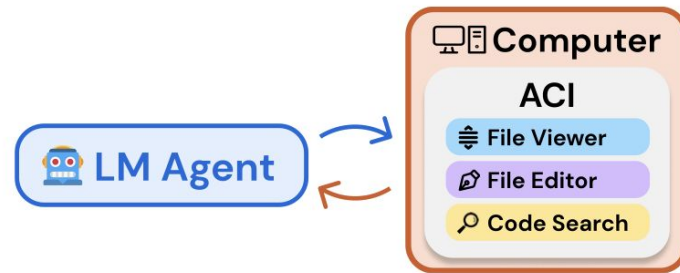
SWE-agent

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, Ofir Press

SWE-agent

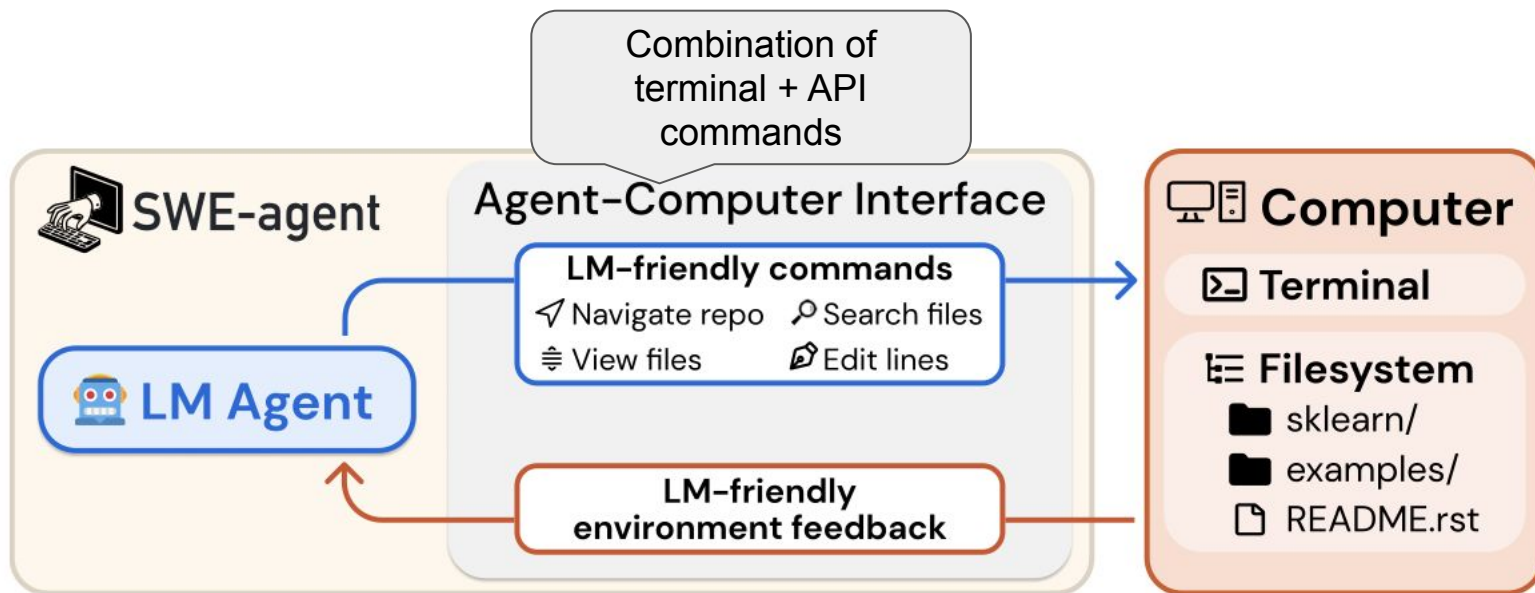


Humans use interfaces to
interact with code



Agents should also be
able to use the same tool

Actions supported by SWE-agent



Results

Model	SWE-bench		SWE-bench Lite	
	% Resolved	\$ Avg. Cost	% Resolved	\$ Avg. Cost
RAG				
w/ GPT-4 Turbo	1.31	0.13	2.67	0.13
w/ Claude 3 Opus	3.79	0.25	4.33	0.25
Shell-only agent				
w/ GPT-4 Turbo	-	-	11.00	1.46
w/o Demonstration	-	-	7.33	0.79
SWE-agent				
w/ GPT-4 Turbo	12.47	1.59	18.00	1.67
w/ Claude 3 Opus	10.46	2.59	13.00	2.18

Agents succeed quickly and fail slowly.

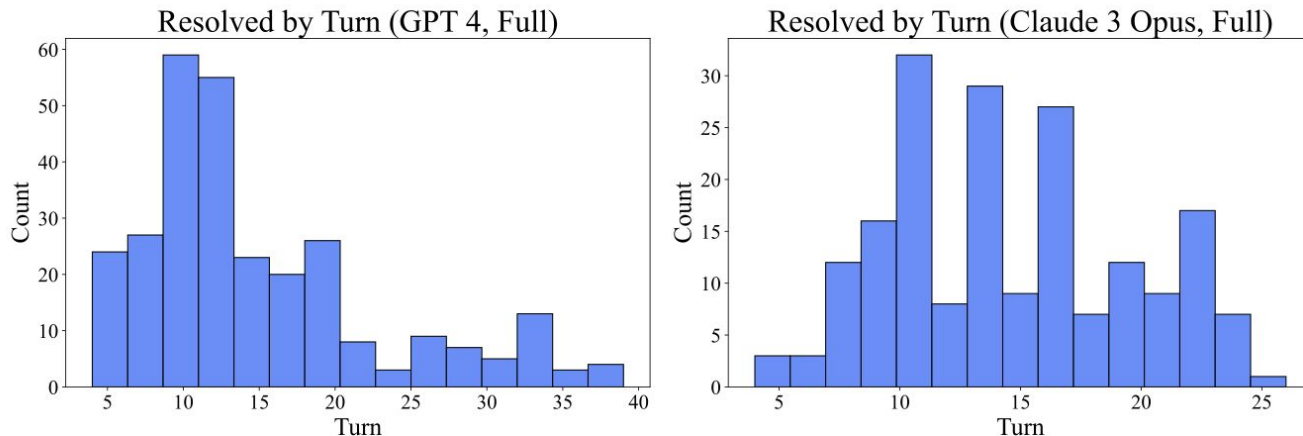


Figure 14: Distribution of the number of turns for interactive trajectories corresponding to solved task instances on SWE-bench. The left histogram shows this distribution for SWE-agent w/ GPT 4 on the full SWE-bench test set (286 trajectories). The right histogram is the performance of SWE-agent w/ Claude 3 Opus on the Lite split of the SWE-bench test set (35 trajectories).

Demo

- Let's use swe agent to repair some code written in Rust
- Clone swe-agent - <https://github.com/SWE-agent/SWE-agent>

Command:

```
python run.py \  
  run \  
  --agent.model.name=claude-3-7-sonnet-20250219 \  
  --env.repo.path=/etc/amp \  
  --problem_statement.path=repair.ps \  
  --env.deployment.image={DOCKER_IMAGE_PATH} \  
  --config={CONFIG_PATH} \  
  --agent.model.per_instance_cost_limit={cost_limit} \  
  --actions.apply_patch_locally=True
```

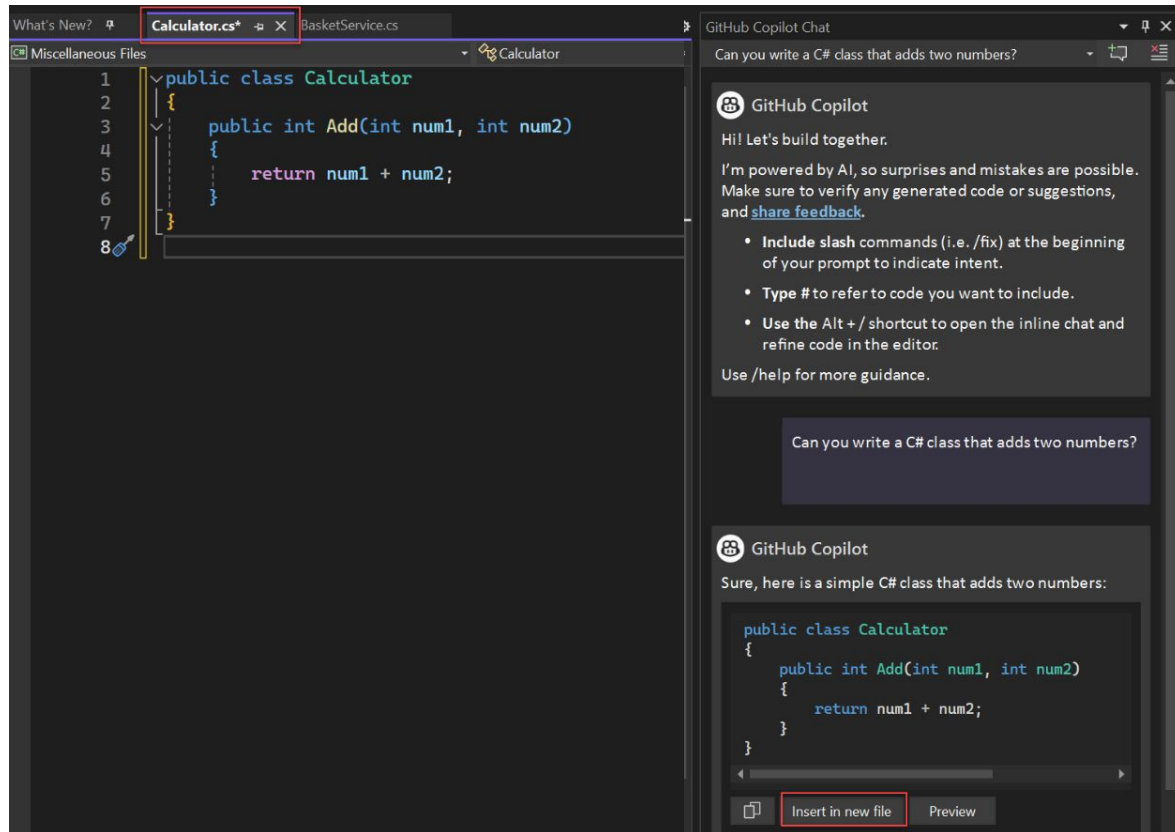

Agents for Code

- Editor based
 - Github Copilot
 - Cursor UI Copilot

```
JS test.js 1 ●
JS test.js >  calculateDaysBetweenDates
1  function calculateDaysBetweenDates(begin, end) {
    var beginDate = new Date(begin);
    var endDate = new Date(end);
    var days = Math.round((endDate - beginDate)
    return days;
    }
2
```

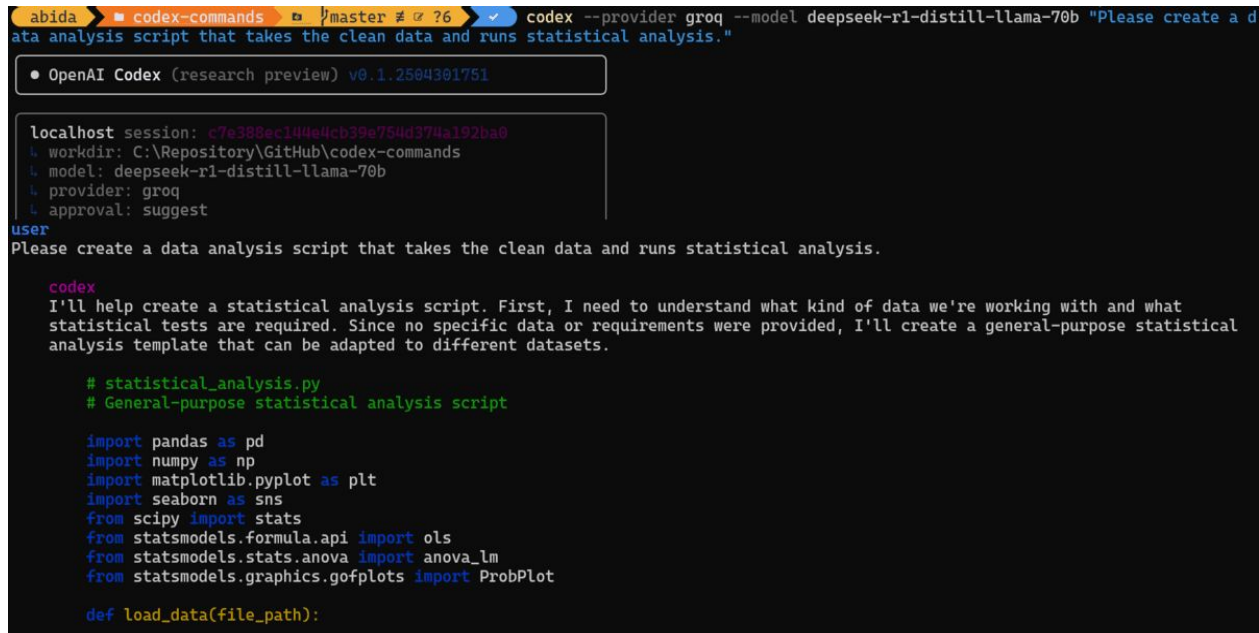
Agents for Code

- Editor based
 - Github Copilot
 - Cursor UI Copilot
- UI Based
 - VS Copilot
 - Cursor Copilot



Agents for Code

- Editor based
 - Github Copilot
 - Cursor UI Copilot
- UI Based
 - VS Copilot
 - Cursor Copilot
- Terminal based
 - SWE-agent
 - Claude Code
 - Codex



```
abida - codex-commands | master # 76 | codex --provider groq --model deepseek-r1-distill-llama-70b "Please create a data analysis script that takes the clean data and runs statistical analysis."
```

```
• OpenAI Codex (research preview) v0.1.2504301751
```

```
localhost session: c7e388ec144e4cb39e754d374a192ba0
└─ workdir: C:\Repository\GitHub\codex-commands
└─ model: deepseek-r1-distill-llama-70b
└─ provider: groq
└─ approval: suggest
```

```
user
Please create a data analysis script that takes the clean data and runs statistical analysis.
```

```
codex
I'll help create a statistical analysis script. First, I need to understand what kind of data we're working with and what statistical tests are required. Since no specific data or requirements were provided, I'll create a general-purpose statistical analysis template that can be adapted to different datasets.
```

```
# statistical_analysis.py
# General-purpose statistical analysis script

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from statsmodels.formula.api import ols
from statsmodels.stats.anova import anova_lm
from statsmodels.graphics.gofplots import ProbPlot

def load_data(file_path):
```